ordering	persistence_id	sequence_number	event_ser_manifest	ever
3	pcontainer-type-key|11	1	example.persistence.SPContainer$CargoAdded	Binary
1	pcontainer-type-key|9	1	example.persistence.SPContainer$CargoAdded	Binary
2	pcontainer-type-key|9	2	example.persistence.SPContainer$CargoAdded	Binary

You've written the journal. Now comes the second part, the projection:

1. Create the tables `akka_projection_offset_store` and `cargos_per _container` using the scripts with the same names in the folder projections by executing the following from the command line in the root directory of the chapter10c project:

```
$ psql -h 127.0.0.1 -d containers -U containers \
    -f projection_offset_store.sql
$ psql -h 127.0.0.1 -d containers -U containers \
    -f projection_cargos_per_container.sql
```

2. Start the projection with only one node by running the following from the command line in the root of the Git repository:

```
$ sbt "chapter10c/runMain example.projection.Main 25525"
```

   - This will create three projections, one per tag. You can see the following output in the projection node:

```
[INFO] [akka.projection.ProjectionBehavior$] [] [containersprojection-
➥akka.actor.default-dispatcher-22] - Starting projection
➥[ProjectionId(CargosPerContainerProjection, container-tag-0)]
[INFO] [akka.projection.ProjectionBehavior$] [] [containersprojection-
➥akka.actor.default-dispatcher-3] - Starting projection
➥[ProjectionId(CargosPerContainerProjection, container-tag-1)]
[INFO] [akka.projection.ProjectionBehavior$] [] [containersprojection-
➥akka.actor.default-dispatcher-5] - Starting projection
➥[ProjectionId(CargosPerContainerProjection, container-tag-2)]
```

Each projection immediately fetches the events from the journal corresponding to its tag and finds that it has no offset stored in `akka_projection_offset_store`. Since there is no offset yet, the projection reads the journal from the beginning until the `akka_projection_offset_store.offset` is equal to the `event_journal .sequence_number`. Again, this is different for each tag.

For each of these events, the application executes the SQL statement from the `CargosPerContainerRepositoryImpl`, shown again here:

```
INSERT INTO cargos_per_container (containerId, cargos) VALUES ($containerId, 1)
      ON CONFLICT (containerId) DO
          UPDATE SET cargos = cargos_per_container.cargos + 1
```

This produces the data in table 10.5. The `akka_projection_offset_store` table is updated to match the `sequence_number` of the event on the journal per container, as shown in table 10.6.